Partie 2 - Chiffrement et déchiffrement (ciphers.py)

```
import string
import unicodedata
from math import gcd

ALPHA = string.ascii_uppercase
CAESAR_ALPHABET = string.ascii_lowercase + string.ascii_uppercase + " "
HILL_ALPHABET = ALPHA + " "
HILL_MOD = len(HILL_ALPHABET)
COMMON_SYMBOLS = " etaoinshrdlu"
PAD = "#"
PLAYFAIR_ALPHABET = ALPHA + string.digits + " .,!?'-:;()/" + PAD
PLAYFAIR_SIZE = 7
```

Outils communs

Les espaces sont chiffrés comme les autres symboles. Les accents sont retirés, puis clean garde seulement les symboles autorisés de chaque algorithme.

```
def remove_accents(text):
    decomposed = unicodedata.normalize("NFD", text)
    return "".join(c for c in decomposed if not unicodedata.combining(c))

def clean(text, alphabet=ALPHA):
    return "".join(c for c in remove_accents(text).upper() if c in alphabet)
```

Chiffrement de César

L'alphabet compte 53 symboles : minuscules, majuscules et espace, en un seul cycle. Chaque symbole est décalé de n rangs vers la droite ou la gauche. Le déchiffrement applique le sens inverse. Deux attaques : essayer les 53 décalages, ou garder le décalage dont le résultat contient le plus de symboles fréquents (espace, e, t, a...).

```
def check_caesar_key(n):
    if n % len(CAESAR_ALPHABET) == 0:
        raise ValueError("clé invalide : un décalage multiple de 53 ne change pas le message")

def caesar(text, n, direction="right"):
    size = len(CAESAR_ALPHABET)
    shift = n % size
    if direction in ("left", "gauche"):
        shift = -shift
    result = ""
    for ch in text:
        if ch in CAESAR_ALPHABET:
            position = CAESAR_ALPHABET.index(ch)
            result += CAESAR_ALPHABET[(position + shift) % size]
        else:
            result += ch
    return result

def caesar_decrypt(text, n, direction="right"):
    if direction in ("left", "gauche"):
        return caesar(text, n, "right")
    return caesar(text, n, "left")

def caesar_bruteforce(text):
    return [(k, caesar(text, k, "left")) for k in range(len(CAESAR_ALPHABET))]

def caesar_score(text):
    score = 0
    for ch in text.lower():
        if ch in COMMON_SYMBOLS:
            score += len(COMMON_SYMBOLS) - COMMON_SYMBOLS.index(ch)
    return score

def caesar_frequency_guess(text):
    shifts = range(len(CAESAR_ALPHABET))
    return max(shifts, key=lambda k: caesar_score(caesar(text, k, "left")))
```

Chiffrement de Hill

L'alphabet compte 27 symboles (A-Z et espace). La clé est une matrice carrée modulo 27. Chaque bloc de n symboles est multiplié par la matrice ; on complète avec des espaces. Pour déchiffrer on utilise l'inverse de la matrice, qui n'existe que si le déterminant est premier avec 27 (non divisible par 3). L'attaque à clair connu retrouve la clé par K = C x P^-1.

```
def det(m):
    if len(m) == 1:
        return m[0][0]
    total = 0
    for j in range(len(m)):
        minor = [row[:j] + row[j + 1:] for row in m[1:]]
        total += (-1) ** j * m[0][j] * det(minor)
    return total

def mat_inverse_mod(m, mod=HILL_MOD):
    n = len(m)
    d = det(m) % mod
    if gcd(d, mod) != 1:
        raise ValueError("la matrice clé n'est pas inversible modulo %d" % mod)
    d_inv = pow(d, -1, mod)
    if n == 1:
        return [[d_inv]]
    inverse = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            minor = [row[:j] + row[j + 1:] for k, row in enumerate(m) if k != i]
            cofactor = (-1) ** (i + j) * det(minor)
            inverse[j][i] = cofactor * d_inv % mod
    return inverse

def check_hill_key(key):
    mat_inverse_mod(key)
    size = len(key)
    identity = all(key[i][j] % HILL_MOD == (1 if i == j else 0) for i in range(size) for j in range(size))
    if identity:
        raise ValueError("clé invalide : la matrice identité ne change pas le message")

def key_from_text(word):
    letters = clean(word, HILL_ALPHABET)
    size = int(len(letters) ** 0.5)
    if size < 2 or size * size != len(letters):
        raise ValueError("la clé doit avoir 4 ou 9 caractères")
    numbers = [HILL_ALPHABET.index(c) for c in letters]
    return [numbers[i * size:(i + 1) * size] for i in range(size)]

def hill_apply(text, key):
    n = len(key)
    numbers = [HILL_ALPHABET.index(c) for c in text]
    result = ""
    for start in range(0, len(numbers), n):
        block = numbers[start:start + n]
        for row in key:
            result += HILL_ALPHABET[sum(a * b for a, b in zip(row, block)) % HILL_MOD]
    return result

def hill_encrypt(text, key):
    mat_inverse_mod(key)
    letters = clean(text, HILL_ALPHABET)
    while len(letters) % len(key) != 0:
        letters += " "
    return hill_apply(letters, key)

def hill_decrypt(text, key):
    inverse = mat_inverse_mod(key)
    letters = clean(text, HILL_ALPHABET)
    if len(letters) % len(key) != 0:
        raise ValueError("la longueur du chiffré doit être un multiple de %d" % len(key))
    return hill_apply(letters, inverse)

def hill_strip_padding(text, n):
```

```
    removed = 0
    while text.endswith(" ") and removed < n - 1:
        text = text[:-1]
        removed += 1
    return text

def block_matrix(text, start, n):
    numbers = [HILL_ALPHABET.index(c) for c in text[start:start + n * n]]
    return [[numbers[i * n + j] for i in range(n)] for j in range(n)]

def mat_mul_mod(a, b):
    size = len(a)
    return [[sum(a[i][k] * b[k][j] for k in range(size)) % HILL_MOD for j in range(size)] for i in range(size)]

def hill_known_plaintext(plain, cipher, n=2):
    plain = clean(plain, HILL_ALPHABET)
    cipher = clean(cipher, HILL_ALPHABET)
    if len(plain) < n * n or len(cipher) < n * n:
        raise ValueError("il faut au moins n*n caractères")
    for start in range(0, len(plain) - n * n + 1, n):
        P = block_matrix(plain, start, n)
        C = block_matrix(cipher, start, n)
        try:
            return mat_mul_mod(C, mat_inverse_mod(P))
        except ValueError:
            continue
    raise ValueError("aucun bloc de clair inversible trouvé")
```

Chiffrement de Playfair

Carré 7x7 de 49 symboles : 26 lettres (I et J séparés), 10 chiffres, espace, ponctuation et # réservé au bourrage. Plus de fusion I/J, et le bourrage # se retire sans ambiguïté. Le texte est coupé en paires (un # sépare deux symboles identiques ou complète la fin). Même ligne : symbole de droite, même colonne : symbole du dessous, sinon on prend les coins du rectangle. Le déchiffrement fait l'inverse. digram_counts sert à l'attaque par fréquence des paires.

```
def check_playfair_key(key):
    if len(set(playfair_text(key)) - {" "}) < 4:
        raise ValueError("clé invalide : il faut au moins 4 caractères différents")

def playfair_text(text):
    return clean(text, PLAYFAIR_ALPHABET).replace(PAD, "")

def playfair_square(key):
    symbols = []
    for ch in playfair_text(key) + PLAYFAIR_ALPHABET:
        if ch not in symbols:
            symbols.append(ch)
    return symbols

def playfair_pairs(text):
    symbols = playfair_text(text)
    pairs = []
    i = 0
    while i < len(symbols):
        first = symbols[i]
        second = symbols[i + 1] if i + 1 < len(symbols) else PAD
        if first == second:
            second = PAD
            i += 1
        else:
            i += 2
        pairs.append((first, second))
    return pairs

def playfair_swap(a, b, square, step):
    size = PLAYFAIR_SIZE
    row_a, col_a = divmod(square.index(a), size)
    row_b, col_b = divmod(square.index(b), size)
    if row_a == row_b:
        col_a = (col_a + step) % size
        col_b = (col_b + step) % size
```

```
    elif col_a == col_b:
        row_a = (row_a + step) % size
        row_b = (row_b + step) % size
    else:
        col_a, col_b = col_b, col_a
    return square[row_a * size + col_a] + square[row_b * size + col_b]

def playfair_encrypt(text, key):
    square = playfair_square(key)
    return "".join(playfair_swap(a, b, square, 1) for a, b in playfair_pairs(text))

def playfair_decrypt(text, key):
    square = playfair_square(key)
    symbols = clean(text, PLAYFAIR_ALPHABET)
    if len(symbols) % 2 != 0:
        raise ValueError("la longueur du chiffré doit être paire")
    return "".join(playfair_swap(symbols[i], symbols[i + 1], square, -1) for i in range(0, len(symbols), 2))

def playfair_strip_padding(text):
    return text.replace(PAD, "")

def digram_counts(text):
    symbols = clean(text, PLAYFAIR_ALPHABET)
    counts = {}
    for i in range(0, len(symbols) - 1, 2):
        pair = symbols[i:i + 2]
        counts[pair] = counts.get(pair, 0) + 1
    return sorted(counts.items(), key=lambda item: -item[1])
```